

Amazon EKS (Elastic Kubernetes Service)

Overview

Amazon EKS (Elastic Kubernetes Service) is a managed Kubernetes service provided by AWS.

EKS simplifies:

- Kubernetes cluster deployment
- Control plane management
- Worker node integration
- Scaling and upgrades
- High availability configuration

AWS manages the Kubernetes control plane, while users manage worker nodes and workloads.

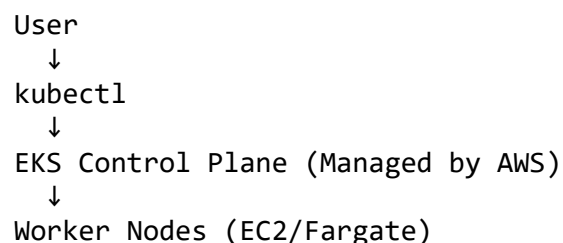
What is eksctl?

eksctl is an official command-line utility for Amazon EKS.

It automates:

- Cluster creation
 - Node group creation
 - IAM integration
 - VPC networking
 - Cluster upgrades
 - Cluster deletion
-

EKS Architecture



↓
Pods

Prerequisites

Before creating an EKS cluster, ensure the following tools are installed:

Tool	Purpose
AWS CLI	Interact with AWS services
eksctl	Create and manage EKS clusters
kubectl	Interact with Kubernetes cluster
AWS IAM Authenticator	IAM authentication for Kubernetes

Install AWS CLI

Official AWS CLI Documentation:

[AWS CLI Installation](#)

Guide <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Verify AWS CLI Installation

Run:

```
aws --version
```

Expected output:

```
aws-cli/2.x.x Python/3.x Linux/x86_64
```

Configure AWS Credentials

Run:

```
aws configure
```

Enter:

- AWS Access Key ID
- AWS Secret Access Key
- AWS Default Region
- Output format (json/yaml)

Example:

```
AWS Access Key ID [None]: AKIAxxxxxxxxx
AWS Secret Access Key [None]: xxxxxxxxxxxxxx
Default region name [None]: ap-south-1
Default output format [None]: json
```

Install eksctl

Run:

```
curl -LO
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_Linux_a
md64.tar.gz"
```

Extract:

```
tar -xzf eksctl_Linux_amd64.tar.gz
```

Move binary:

```
sudo mv eksctl /usr/local/bin
```

Verify eksctl Installation

Run:

```
eksctl version
```

Example output:

```
0.191.0
```

Install kubectl (Kubernetes CLI)

Run:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Install kubectl:

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Verify kubectl Installation

Run:

```
kubectl version --client
```

Expected output:

Client Version: v1.xx.x

Install AWS IAM Authenticator (Optional)

Some environments may require AWS IAM Authenticator.

Run:

```
curl -O aws-iam-authenticator https://amazon-eks.s3.us-west-  
2.amazonaws.com/1.15.10/2020-02-22/bin/linux/amd64/aws-iam-authenticator
```

Note:

Replace version if required.

Make Binary Executable

```
chmod +x ./aws-iam-authenticator
```

Move Binary to System Path

```
sudo mv ./aws-iam-authenticator /usr/local/bin
```

Verify AWS IAM Authenticator

Run:

```
aws-iam-authenticator version
```

Expected output:

```
{"Version":"1.15.10", "Commit":"abcdef123456"}
```

EKS Cluster Creation

Step 1: Create an EKS Cluster Using eksctl

Run the following command:

```
eksctl create cluster \
--name my-cluster \
--region ap-south-1 \
--nodegroup-name my-nodes \
--node-type t3.medium \
--nodes 1
```

Explanation of Parameters

Parameter	Description
--name	EKS cluster name
--region	AWS region
--nodegroup-name	Worker node group name
--node-type	EC2 instance type
--nodes	Number of worker nodes

What eksctl Creates Automatically

The command provisions:

- EKS Control Plane
- VPC
- Subnets

- Security Groups
 - IAM Roles
 - Worker Nodes
 - CloudFormation Stack
 - Auto Scaling Group
-

Step 2: Verify Cluster Creation

Cluster provisioning usually takes:

15–20 minutes

Check cluster status:

```
eksctl get cluster --region ap-south-1
```

Example output:

NAME	REGION	EKCTL CREATED
my-cluster	ap-south-1	True

Step 3: Update kubeconfig

Configure kubectl to connect to EKS cluster.

Run:

```
aws eks update-kubeconfig --region ap-south-1 --name my-cluster
```

Expected output:

```
Added new context arn:aws:eks:ap-south-1:xxxxxxx:cluster/my-cluster to  
/home/ubuntu/.kube/config
```

Step 4: Validate Cluster Connection

Run:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-xx-xx.ap-south-1.compute.internal	Ready	<none>	10m	v1.30.x

Step 5: Deploy a Sample Application

Deploy Nginx application.

Create Deployment

Run:

```
kubectl create deployment nginx --image=nginx
```

Verify Deployment

```
kubectl get deployment
```

Expose Deployment Using LoadBalancer Service

Run:

```
kubectl expose deployment nginx --type=LoadBalancer --port=80
```

This creates an AWS Elastic Load Balancer automatically.

Verify Service

Run:

```
kubectl get svc
```

Example output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S)			
nginx	LoadBalancer	10.100.x.x	a1b2c3d4e5.ap-south-1.elb.amazonaws.com
			80:xxxxx/TCP

Access the Application

Open:

`http://<EXTERNAL-IP>`

You should see:

Welcome to nginx!

Step 6: Check Cluster Resources

View Nodes

```
kubectl get nodes
```

View Pods

```
kubectl get pods
```

View Services

```
kubectl get svc
```

View All Resources

```
kubectl get all
```

Step 7: Scaling Worker Nodes

Scale worker nodes using eksctl.

Example:


```
eksctl scale nodegroup \  
--cluster=my-cluster \  
--nodes=3 \  
--name=my-nodes \  
--region=ap-south-1
```

Step 8: Delete EKS Cluster

To avoid unnecessary AWS charges, delete unused clusters.

Run:

```
eksctl delete cluster --name my-cluster --region ap-south-1
```

EKS Networking Components

Component	Purpose
VPC	Network isolation
Subnets	Public/private networking
Security Groups	Firewall rules
Load Balancer	External traffic routing
ENI	Pod networking

EKS Node Types

EC2 Worker Nodes

Traditional Kubernetes worker nodes using EC2 instances.

AWS Fargate

Serverless Kubernetes execution environment.

Benefits:

- No node management
- Pay per pod usage

- Simplified operations
-

Common EKS Commands

List Clusters

```
eksctl get cluster
```

List Node Groups

```
eksctl get nodegroup --cluster=my-cluster --region=ap-south-1
```

Describe Cluster

```
aws eks describe-cluster --name my-cluster --region ap-south-1
```

Check kubeconfig Context

```
kubectl config current-context
```

Troubleshooting

kubectl Cannot Connect to Cluster

Run:

```
aws eks update-kubeconfig --region ap-south-1 --name my-cluster
```

Unauthorized Error

Check:

- AWS credentials

- IAM permissions
 - kubeconfig
-

Worker Nodes Not Joining Cluster

Check:

- Security groups
 - IAM node role
 - VPC networking
 - CloudFormation stack events
-

LoadBalancer External IP Pending

Check:

- Public subnets
 - Internet Gateway
 - AWS Load Balancer Controller
 - Service type
-

Best Practices

Use Separate Node Groups

Example:

Node Group	Purpose
system-nodes	Kubernetes system workloads
app-nodes	Application workloads
monitoring-nodes	Monitoring stack

Enable Cluster Autoscaler

Automatically scale worker nodes.

Use IAM Roles for Service Accounts (IRSA)

Avoid storing AWS credentials inside pods.

Use Managed Node Groups

AWS-managed node lifecycle simplifies operations.

Enable Monitoring

Recommended tools:

- Prometheus
 - Grafana
 - CloudWatch
 - AWS Container Insights
-

Recommended EKS Components

Component	Purpose
HPA	Scale pods
VPA	Optimize resources
Cluster Autoscaler	Scale worker nodes
Ingress Controller	Manage external traffic
Metrics Server	Resource metrics

Summary

Amazon EKS:

- Simplifies Kubernetes deployment on AWS
- Provides managed Kubernetes control plane
- Integrates with AWS networking and IAM
- Supports autoscaling and load balancing

- Enables production-grade Kubernetes deployments